

ML course

Lecture 09:

Neural Networks basics

Iurii Efimov



Outline

1. Neural Networks in different areas.
Historical overview.
2. Backpropagation.
3. More on backpropagation.
4. Activation functions.
5. Playground.

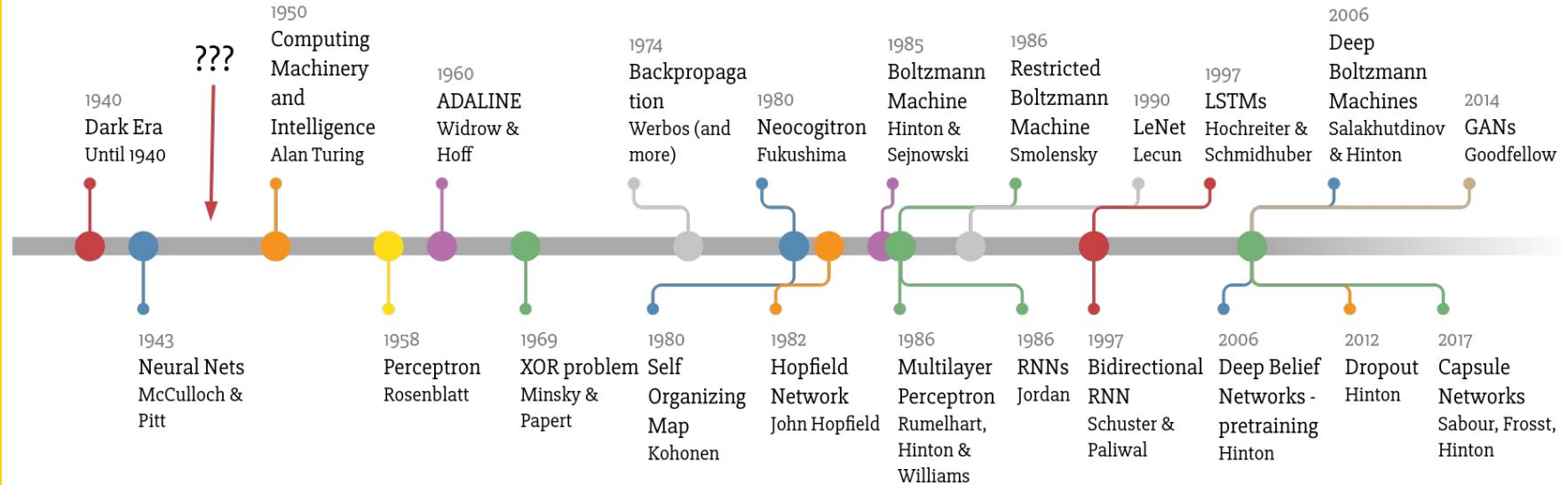
History of Deep Learning

girafe
ai

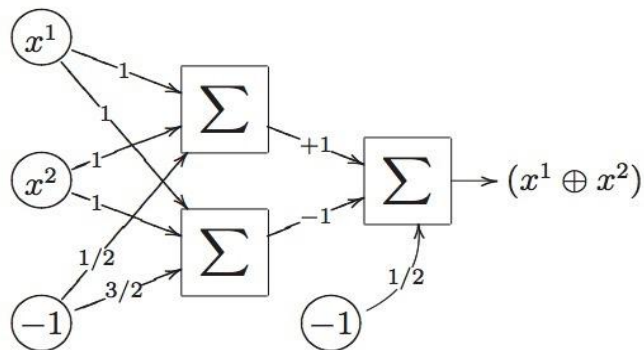
01



Deep Learning Timeline

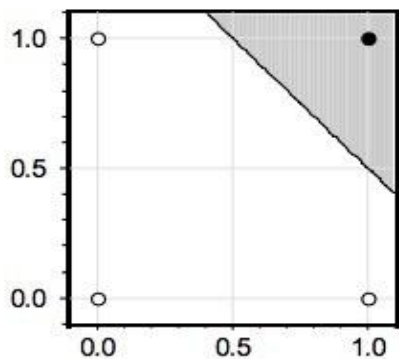


XOR problem

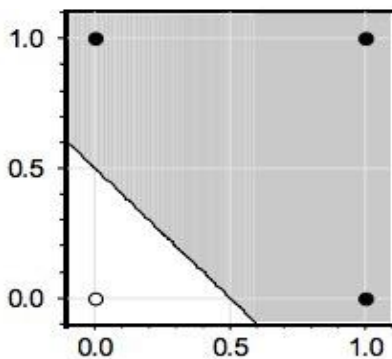


This 2-layer NN (on the left) implements XOR with only x^1 and x^2 features.

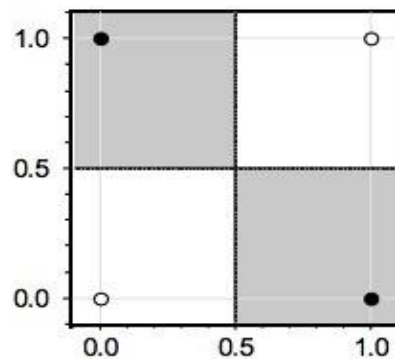
1-layer NN also can succeed, but only with extra feature $x^1 * x^2$.



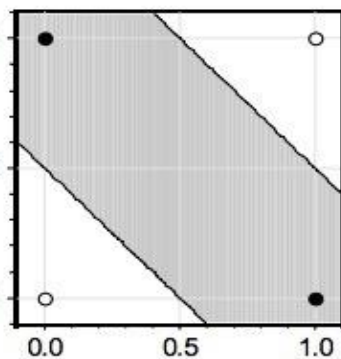
AND



OR



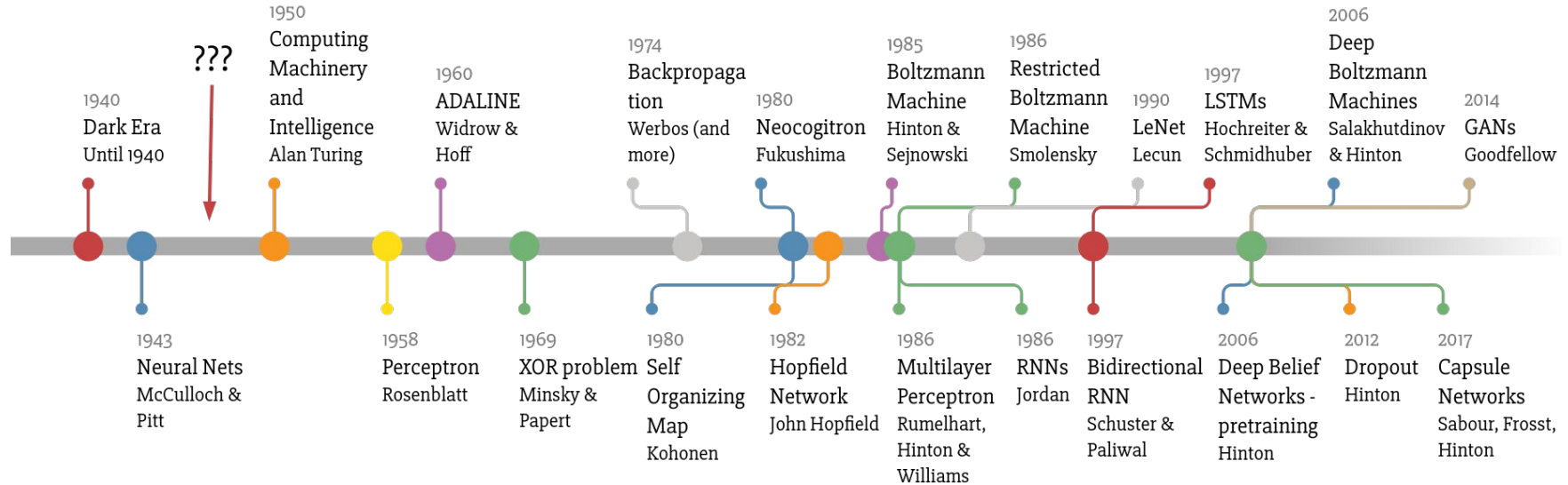
XOR(with $x^1 * x^2$)



XOR

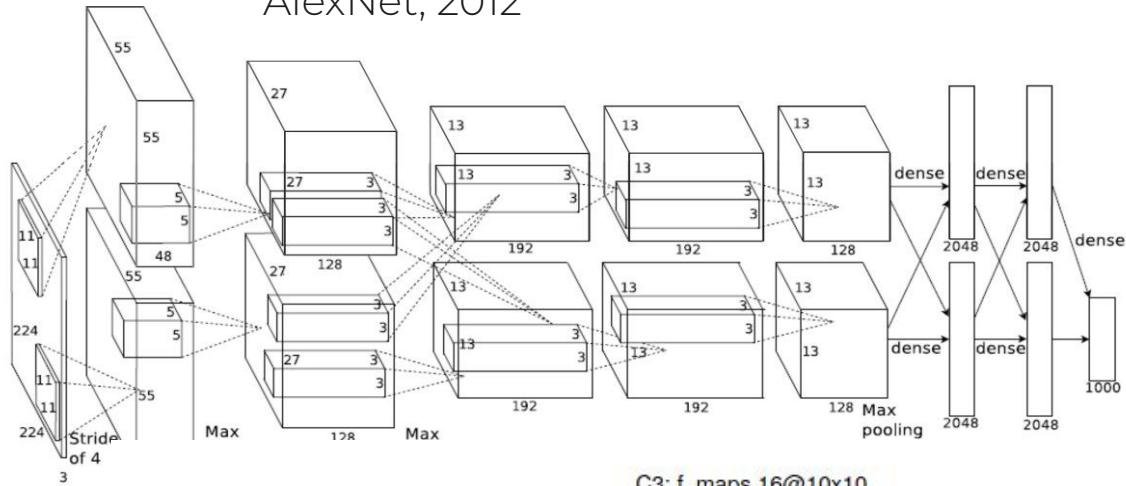


Deep Learning Timeline

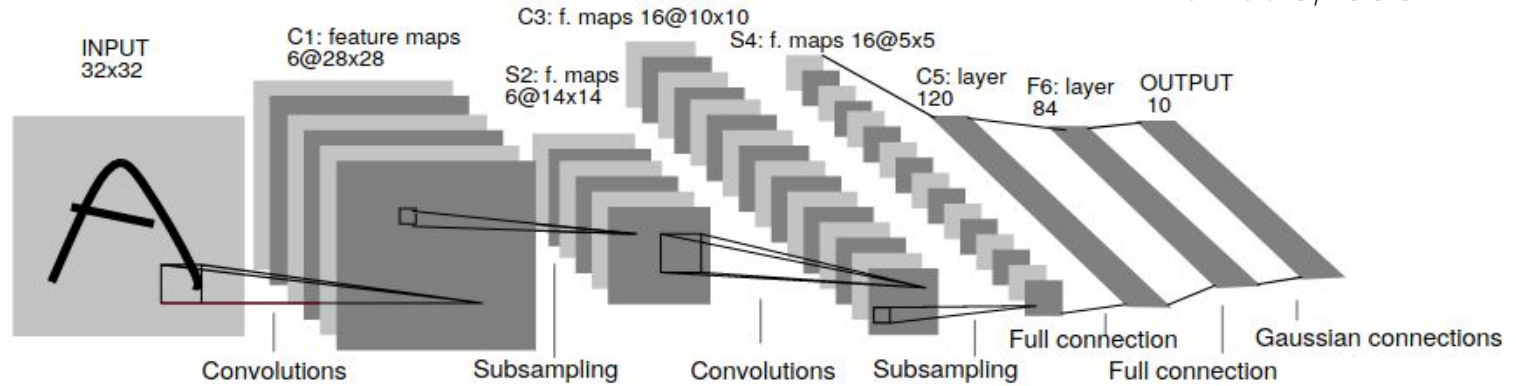




AlexNet, 2012

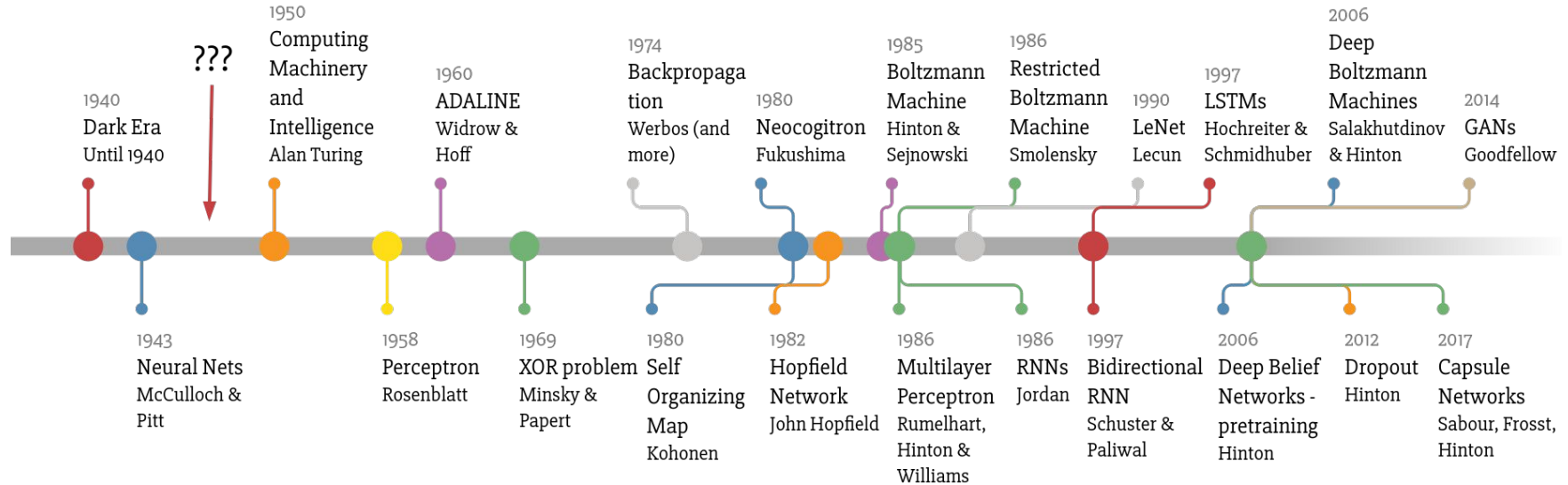


LeNet-5, 1998





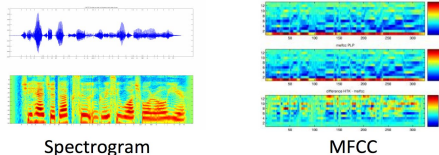
Deep Learning Timeline



Real world applications



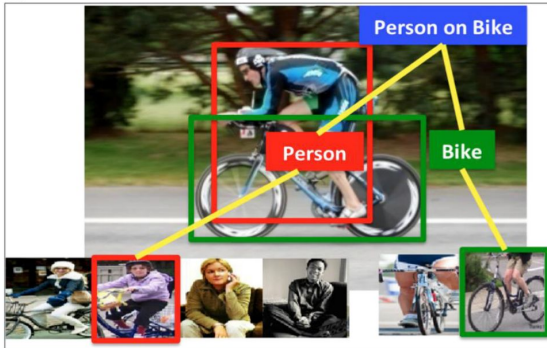
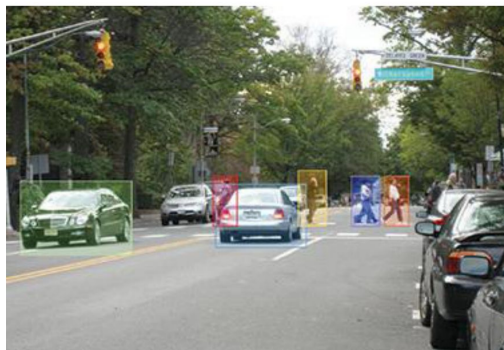
Audio Features



Spectrogram

MFCC

- Object detection
- Action classification
- Image captioning
- ...



"man in black shirt is playing guitar."

GANs. 2014+



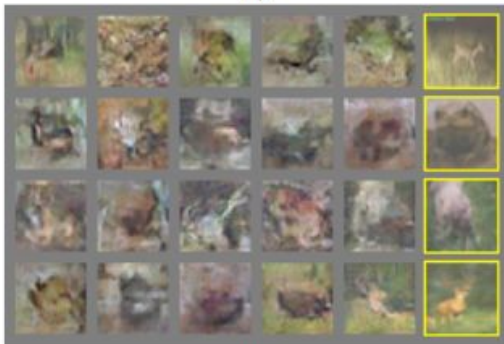
a)



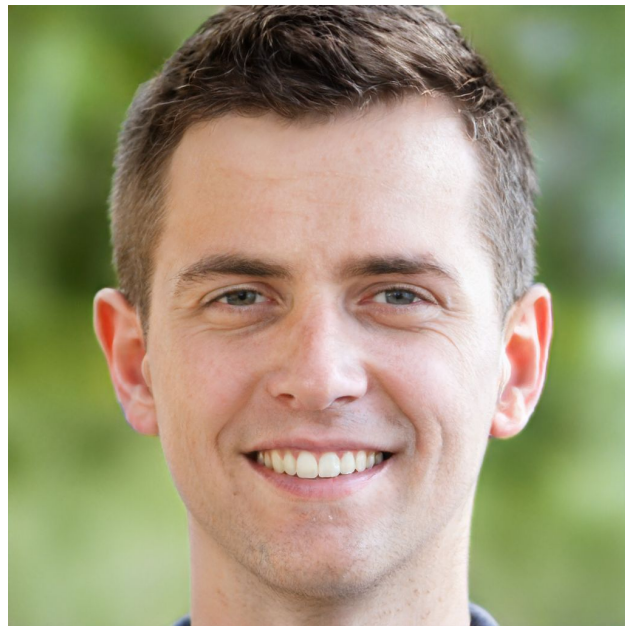
b)



c)



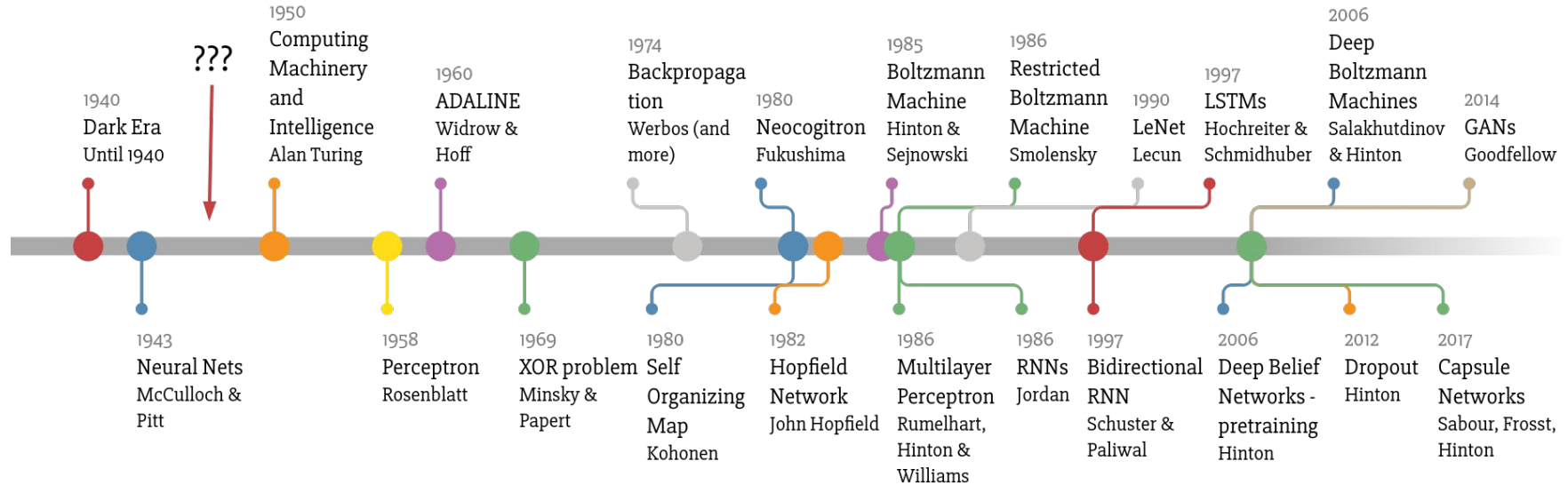
d)



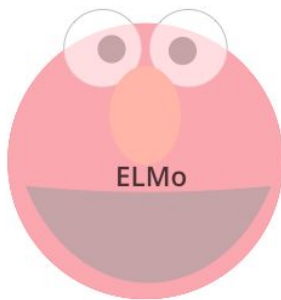
<https://thispersondoesnotexist.com/>



Deep Learning Timeline

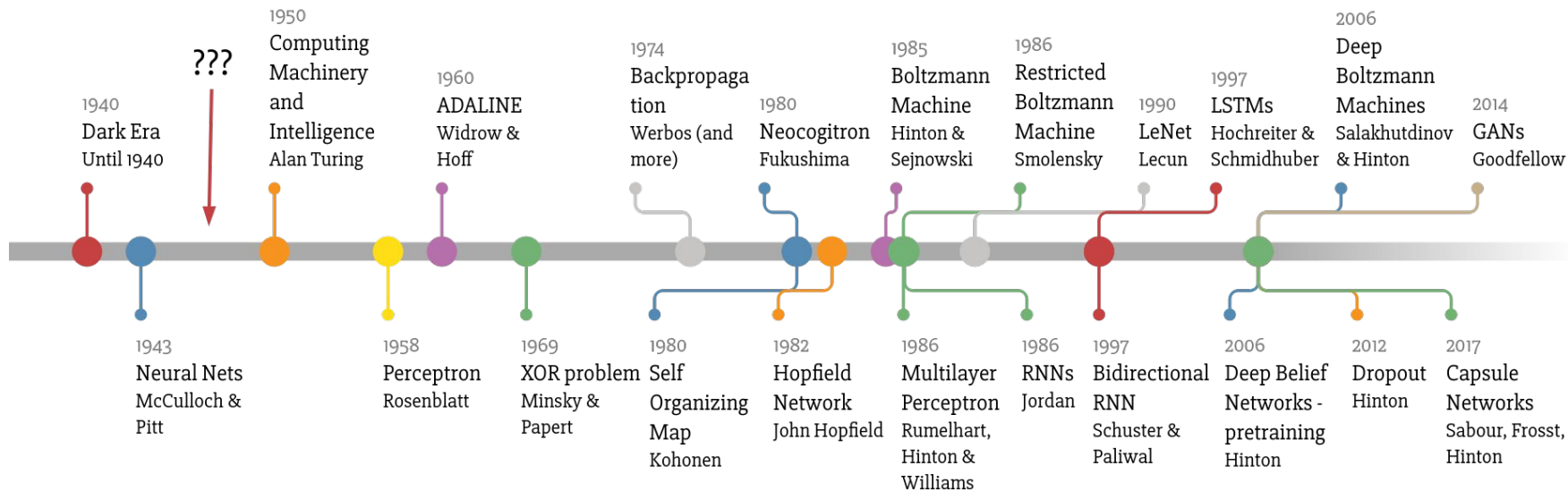


Transformer, BERT, GPT-2 and more, 2017+





Deep Learning Timeline





Term origin

Artificial neural networks were originally used to model biological neural networks starting in the 1930s under the approach of connectionism.

However, starting with the invention of the perceptron, a simple artificial neural network, by Warren McCulloch and Walter Pitts in 1943, followed by the implementation of one in hardware by Frank Rosenblatt in 1957, artificial neural networks became increasingly used for machine learning applications instead, and **increasingly different from their biological counterparts**.

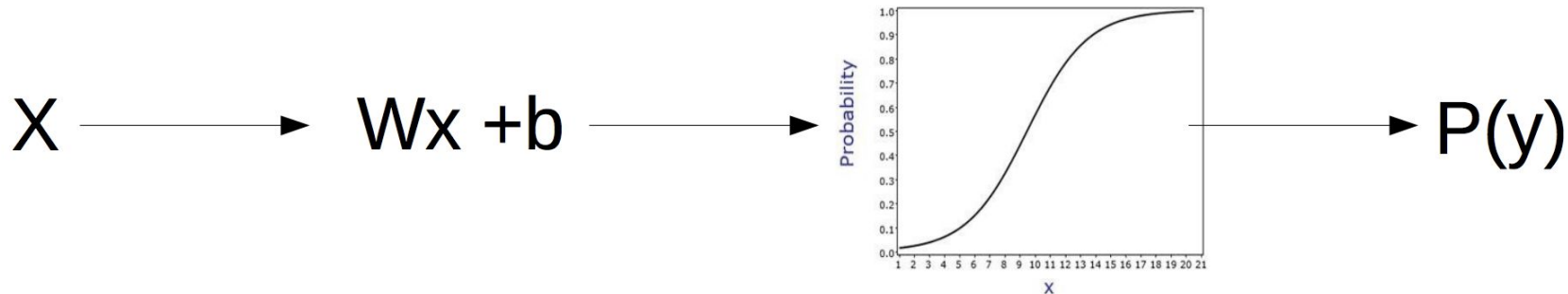
[Common knowledge site](#)

Deep Learning: intuition

girafe
ai

02

Logistic regression



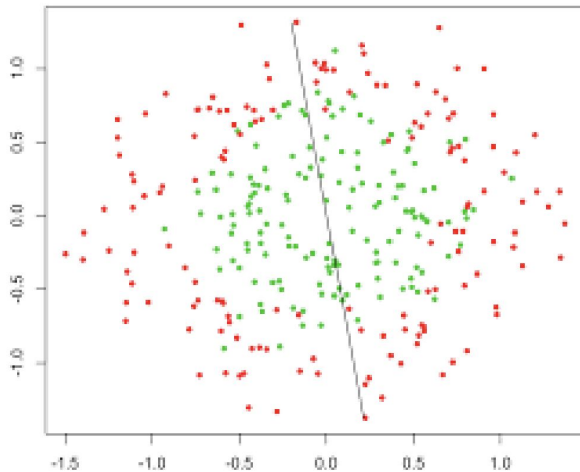
$$P(y|x) = \sigma(w \cdot x + b)$$

$$L = - \sum_i y_i \log P(y|x_i) + (1 - y_i) \log (1 - P(y|x_i))$$

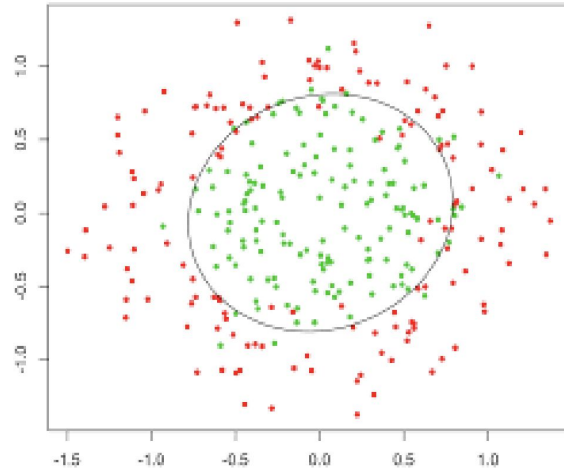
Problem: nonlinear dependencies



Logistic regression (generally, linear model) need feature engineering to show good results.



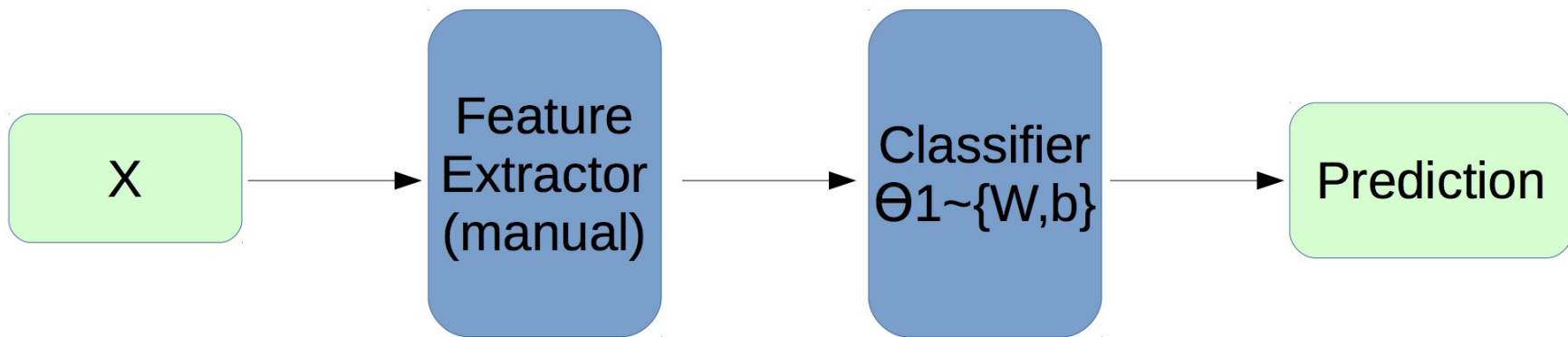
What we have



What we want

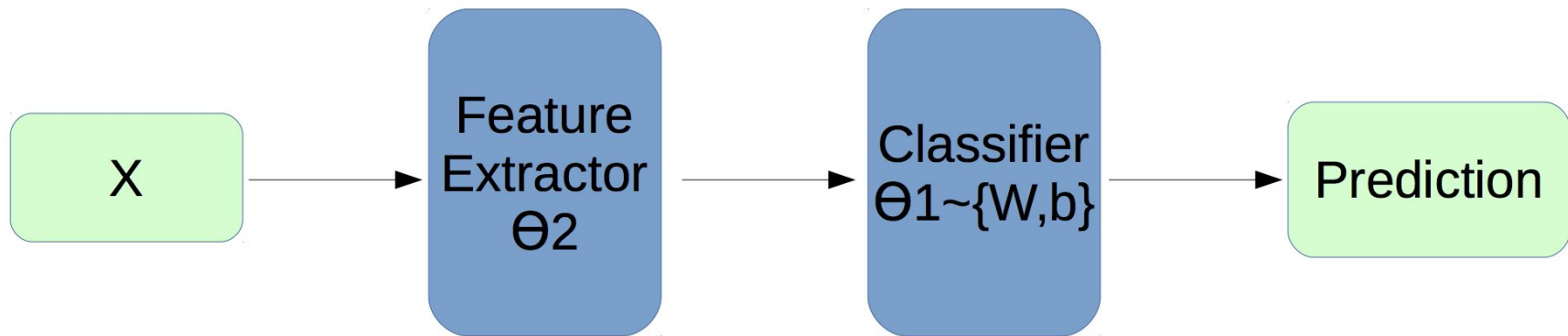
And feature engineering is an art.

Classic pipeline



Handcrafted features, generated by experts.

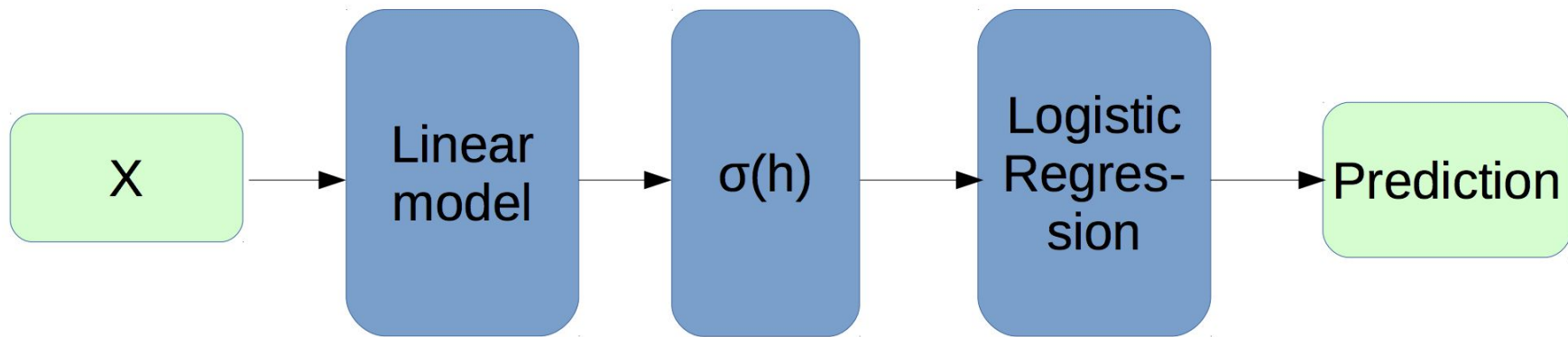
NN pipeline



Automatically extracted features.



NN pipeline: example



E.g. two logistic regressions one after another.

Actually, it's a neural network.



Graphical layer notation

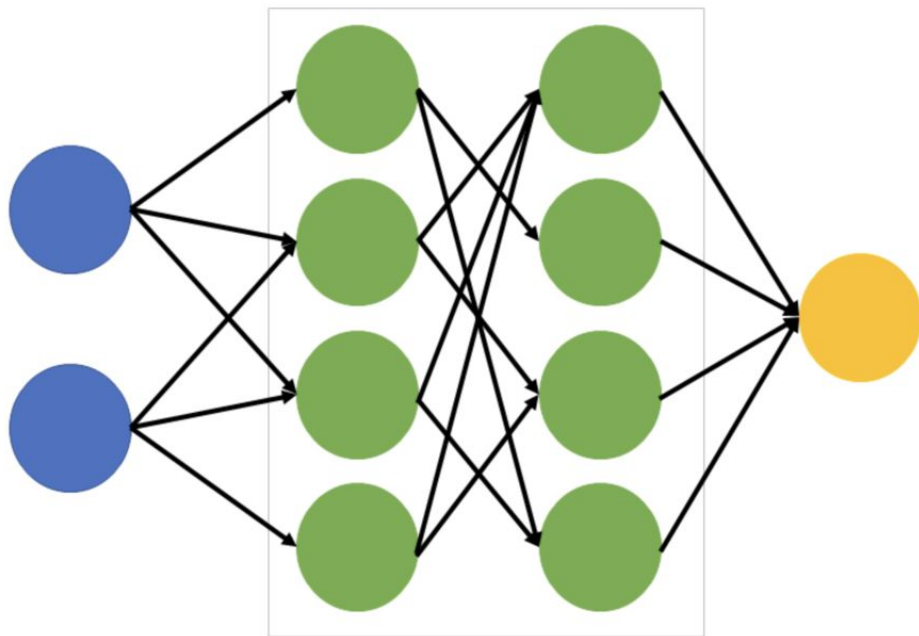
Activation functions are assumed implicitly.

How many layers are presented on the picture?

Input layer

Hidden layer

Output layer



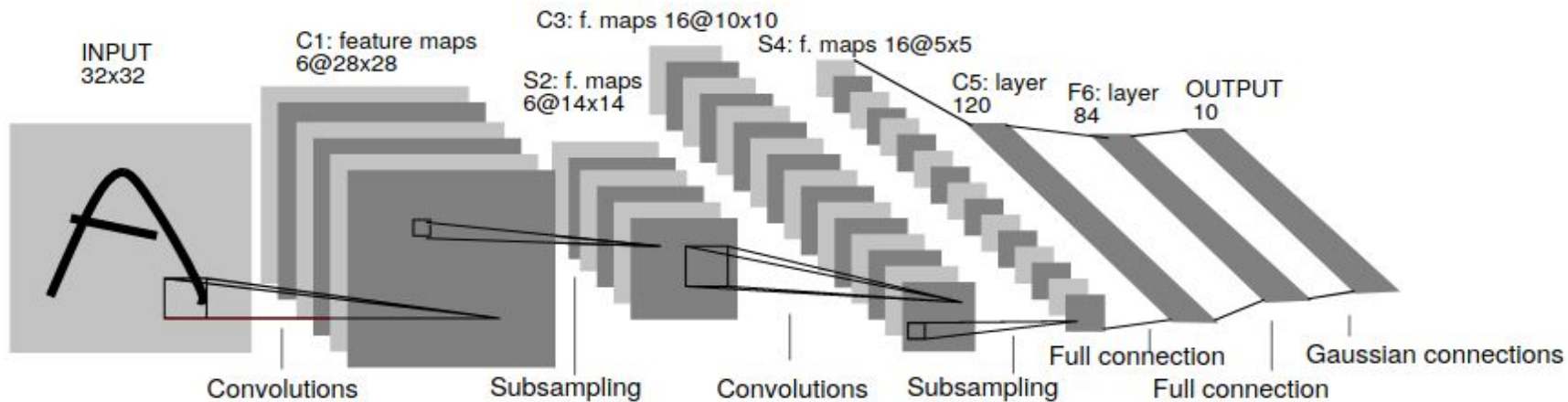
3 layers.... 🧐

Layer is set of
arrows 🧐🧐🧐



Graphical layer notation

How many layers are presented on the picture?



5 layers....



Layer notations

First notation **counts only trainable transforms**.

In such way [LeNet-5](#) has 5 trainable layers. It is used in academical papers and as consequence in popular architectures such as VGG-19, ResNet-101.

Layers are: Linear, Convolutional, Self-attention, etc.

This is main notation used in everyday life.

Second notation **counts every transform** made in neural network.

It is derived from engineering practice such as PyTorch code where each Module added to Sequential container is considered as a layer.

Layers are: all from first notation, Sigmoid, Dropout, BatchNorm, etc.

Third notation **counts feature vectors** in between transforms.

Usually such layers are not counted, it is just a notation to indicate parts of neural network.

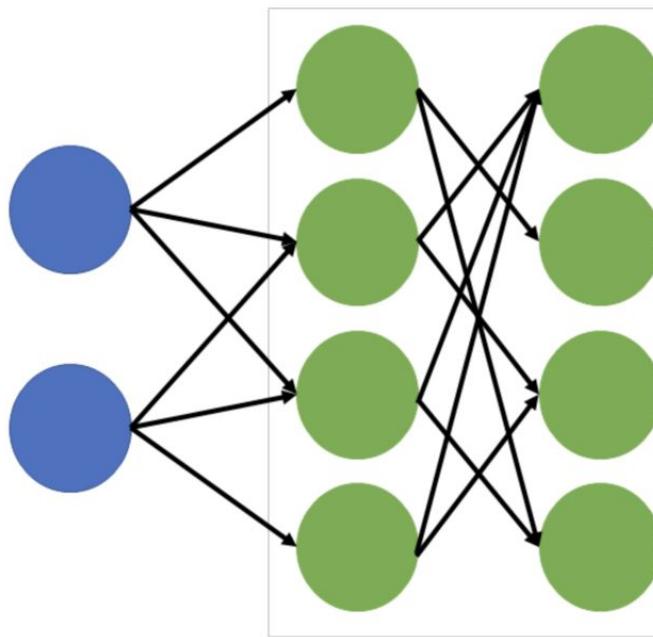
Layers are: Input, Output and Hidden.

Embedding



in ML is a representation of some object in real valued linear space.

Often embedding are just output of one but last layer of a neural network. Such network is called backbone.



Embedding to $\mathbb{R}^4$

Batches



are (usually small) subsamples of dataset that are processed in parallel and used to calculate gradient for one step of gradient descent techniques.

also known as **mini-batches** which is a full synonym of regular batches

Linear layer

girafe
ai

03

Linear layer



It is just a linear transform of input features into output features.

This layer also called **Dense** or **Fully Connected (FC)** because each input feature may affect each output feature.

Also it is sometimes called **Perceptron** or **Multi-layer perceptron (MLP)** in case of multiple linear layers. Another name is **Feedforward neural network**.



Linear layer formula

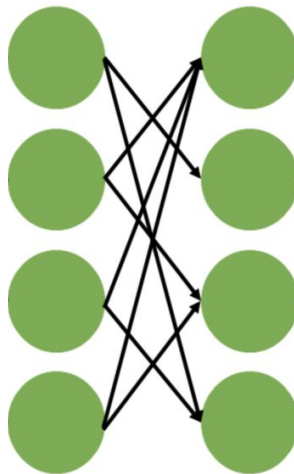
Implementation:

<https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>

Number of parameters:

`in*out + out`

[broadcasting rules](#)



$$Z = X \cdot W + b$$

$$Z \in R^{n \times \text{out}}$$

$$X \in R^{n \times \text{in}}$$



Main layers in NNs

1. Linear layer
2. RNN
 - a. LSTM, GRU
3. CNN
4. Attention
 - a. Cross-attention, self-attention, etc...

Actually all of them are just fancy wrappers around *Linear layer*...

We will cover them one by one in the next lectures.



Weights initialization

1. Zero
 - a. brings zero output
2. Constant (non-zero)
 - a. same gradient for all weights
3. Uniform / Normal
 - a. $-1; 1 / 0; 1$
 - b. mean and variance explode
4. Uniform / Normal
 - a. $-1/\sqrt{\text{in}}; 1/\sqrt{\text{in}} / 0; 1/\sqrt{\text{in}}$
 - b. ok

$$Z = X \cdot W + b$$

$$Z \in R^{n \times \text{out}}$$

$$X \in R^{n \times \text{in}}$$

Universal approximation theorem



state that feedforward networks with a single hidden layer can act as a "universal approximator." Universality is achieved by increasing the number of neurons in the hidden layer, making the network "wider" or "deeper."

These are existence theorems. They guarantee that a network with the right structure exists, but they do not provide a method for finding the network's parameters (training it), nor do they specify exactly how large the network must be for a given function. Finding a suitable network remains a practical challenge that is typically addressed with optimization algorithms like backpropagation.

https://en.wikipedia.org/wiki/Universal_approximation_theorem

Activation functions

girafe
ai

04

Definition



Activation function is a function:

1. non-linear
2. differentiable
 - a. excluding Null set points

Nonlinearities

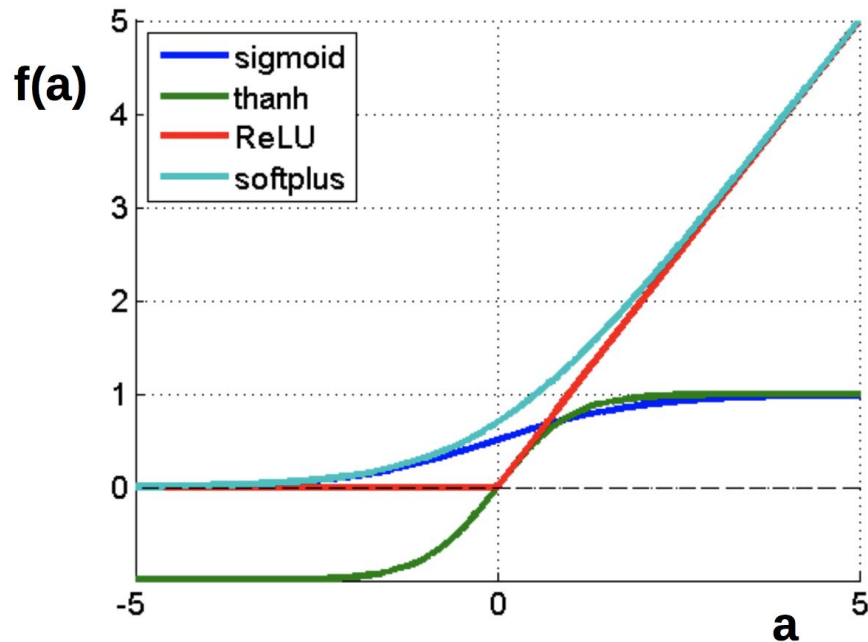


$$f(a) = \frac{1}{1 + e^{-a}}$$

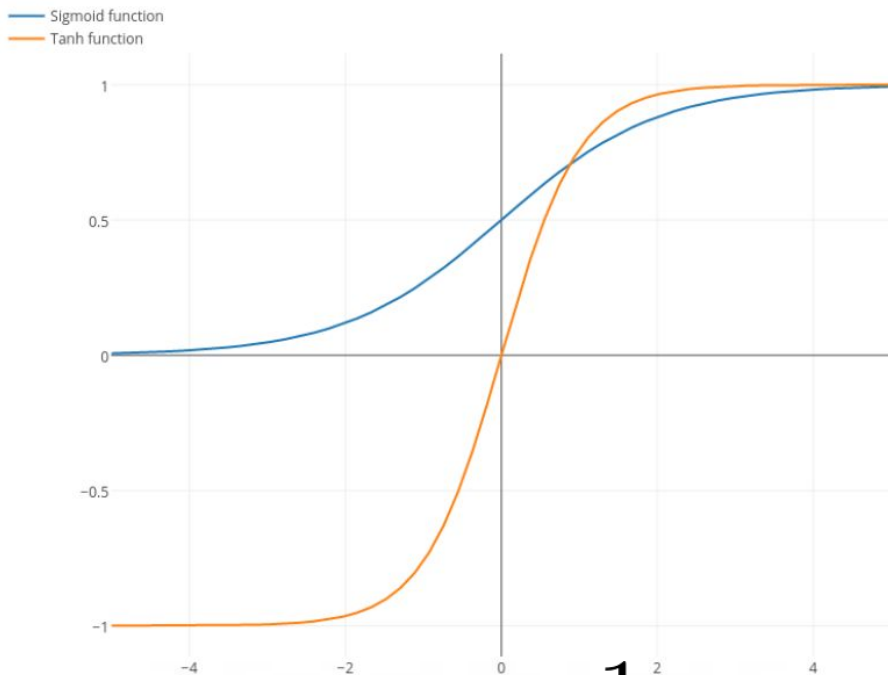
$$f(a) = \tanh(a)$$

$$f(a) = \max(0, a)$$

$$f(a) = \log(1 + e^a)$$



Sigmoid



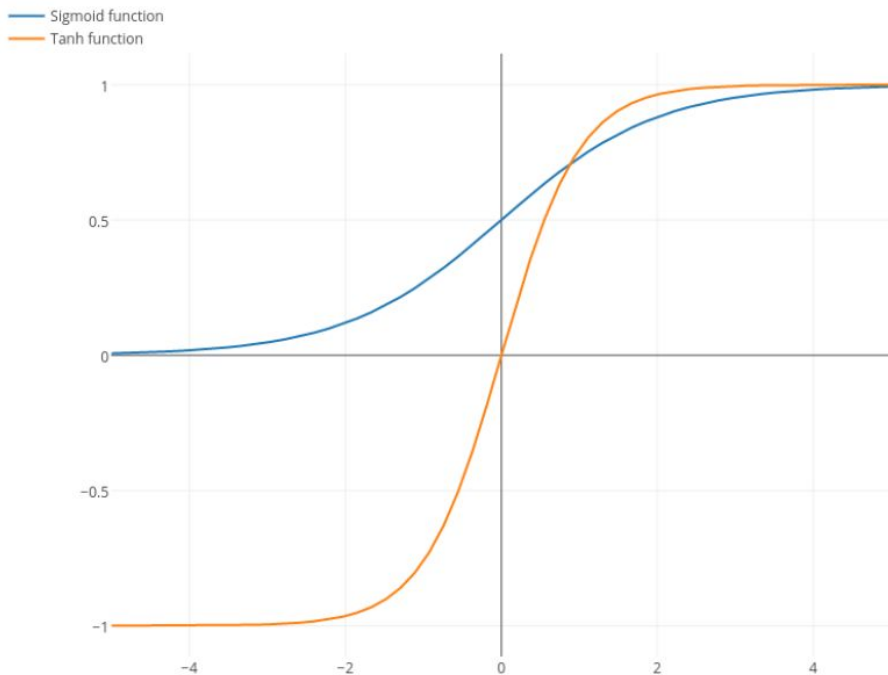
$$f(a) = \frac{1}{1 + e^{-a}}$$

- Maps $\mathbb{R}$ to $(0,1)$
- Historically popular, one of the first approximations of neuron activation

Problems:

- Almost zero gradients on the both sides (saturation)
- Shifted (not zero-centered) output
- Expensive computation of the exponent

tanh



$$f(a) = \tanh(a)$$

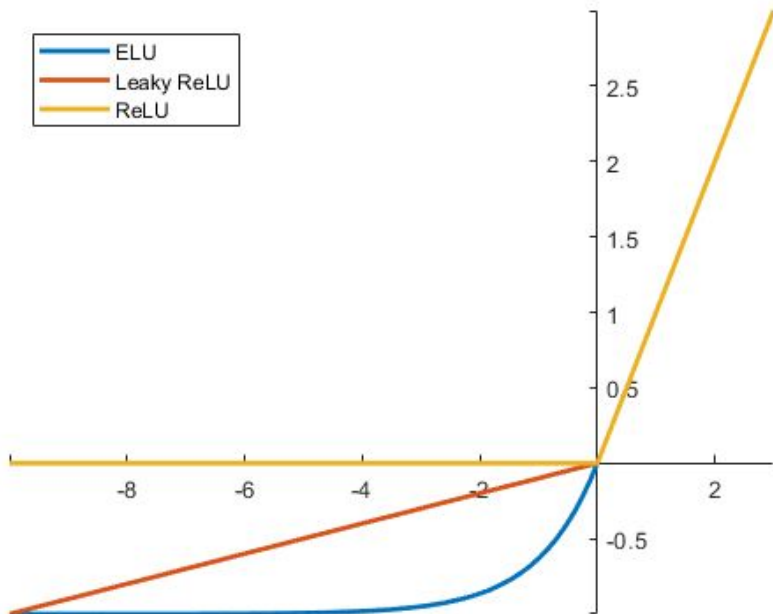


- Maps $\mathbb{R}$ to $(-1,1)$
- Similar to the Sigmoid in other ways

Problems:

- Almost zero gradients on the both sides (saturation)
- ~~Shifted (not zero centered) output~~
- Expensive computation of the exponent

Rectified Linear Unit (ReLU)



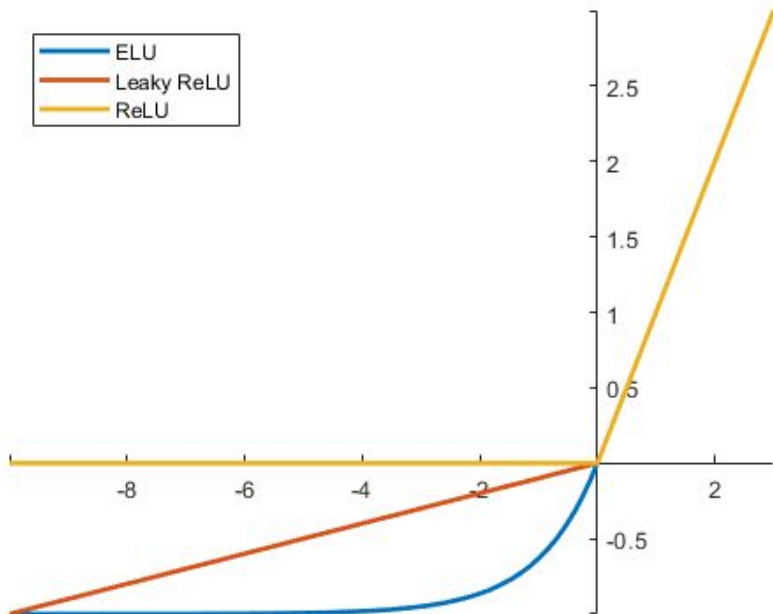
- Very simple to compute (both forward and backward)
 - Up to 6 times faster than Sigmoid
- Does not saturate when $x > 0$
 - So the gradients are not 0

Problems:

- Zero gradients when $x < 0$
- Shifted (not zero-centered) output

$$f(a) = \max(0, a)$$

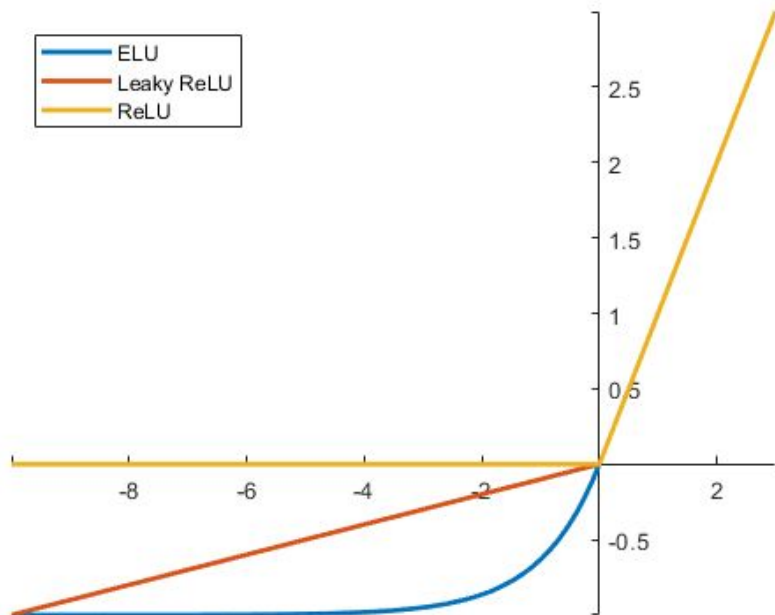
LeakyReLU



- Very simple to compute (both forward and backward)
Up to 6 times faster than Sigmoid
- Does not saturate
- Shifted, but not so much

$$f(a) = \max(0.01a, a)$$

ELU



- Similar to ReLU
- Does not saturate
- Close to zero mean outputs

Problems:

- Requires exponent computation

$$f(a) = \begin{cases} a, & a > 0 \\ \alpha(\exp(a) - 1), & a \leq 0 \end{cases}$$

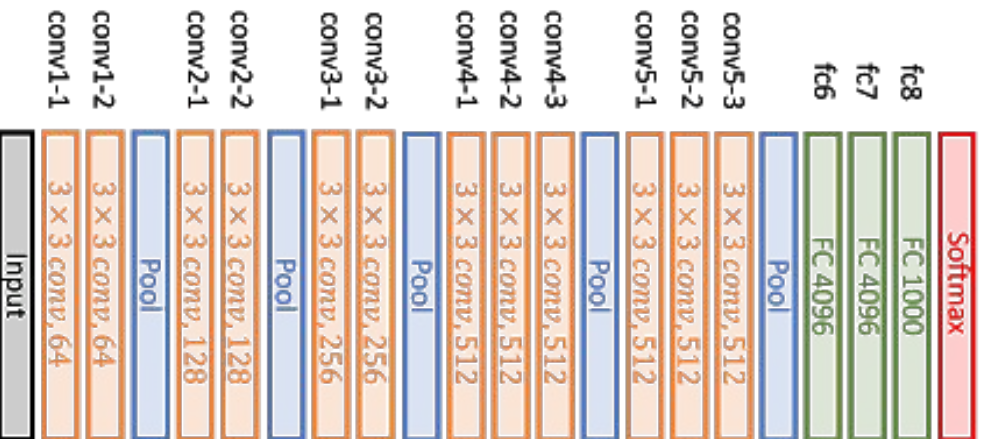
Activation functions sum up



- Use ReLU as baseline approach
 - if not satisfied try change data or model architecture
 - if you want to perform research - try Leaky ReLU, ELU, etc...
- Sigmoid and tanh are used in special cases tanh such as RNN
 - Do not use Sigmoid except last layer in generic case



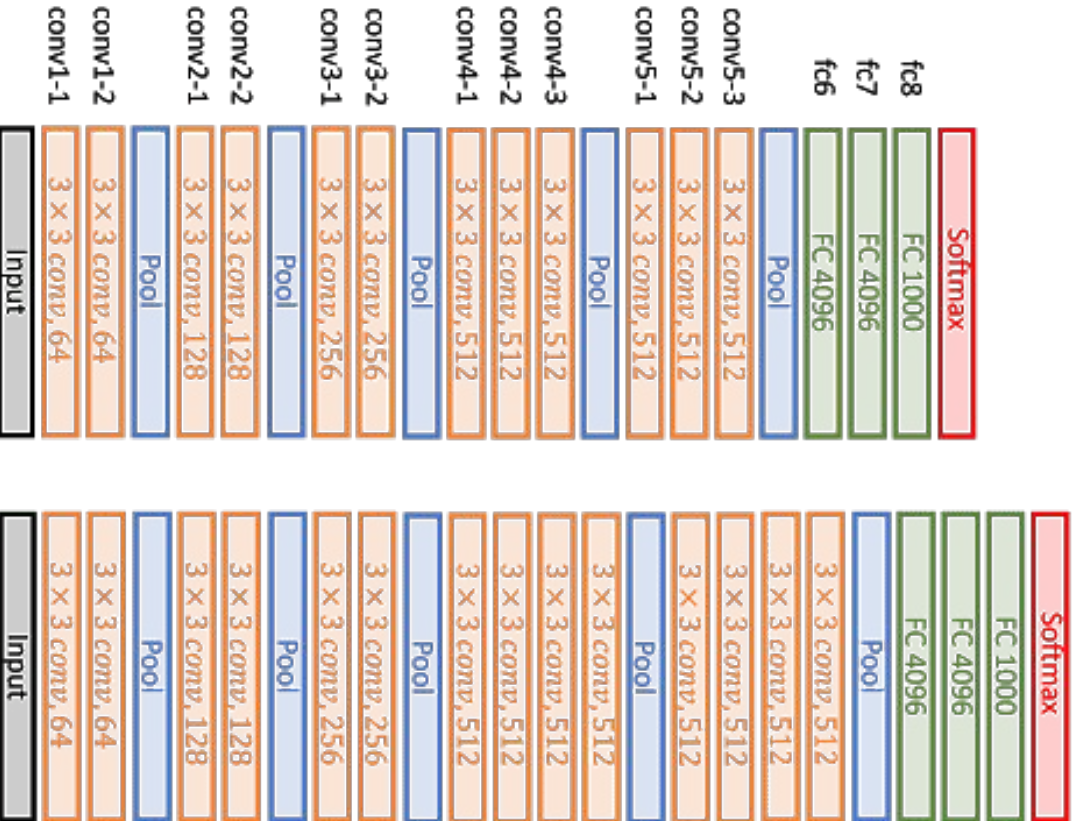
Actually, networks can be deep



VGG16



And deeper...



Backpropagation

girafe
ai

05

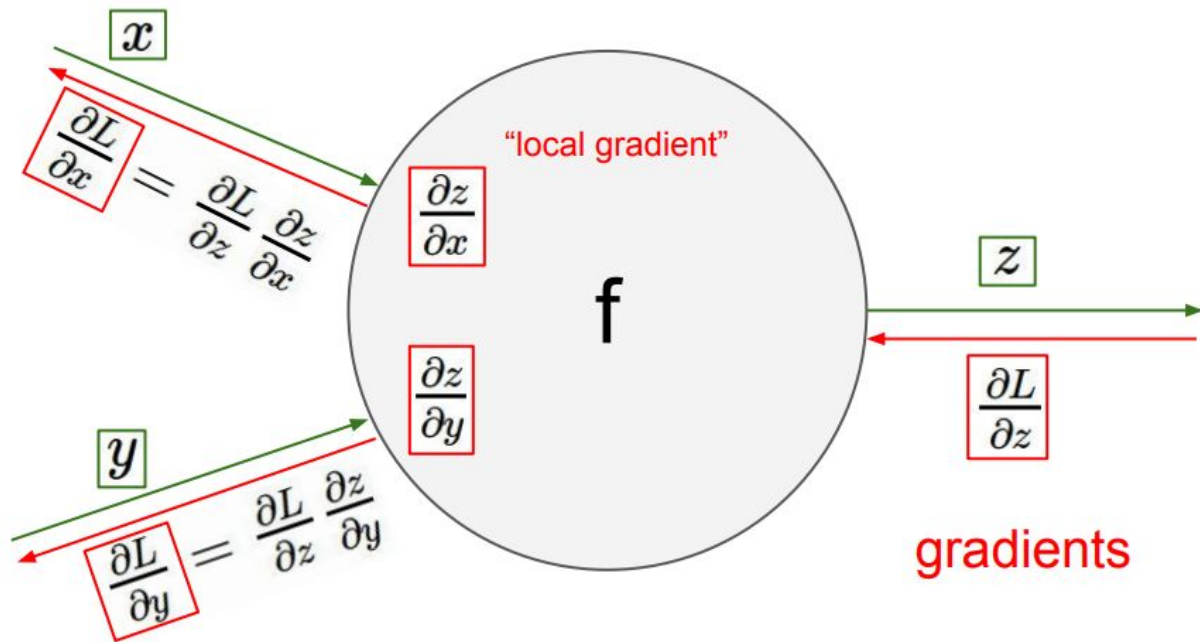
Backpropagation and chain rule



Chain rule is just simple math:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

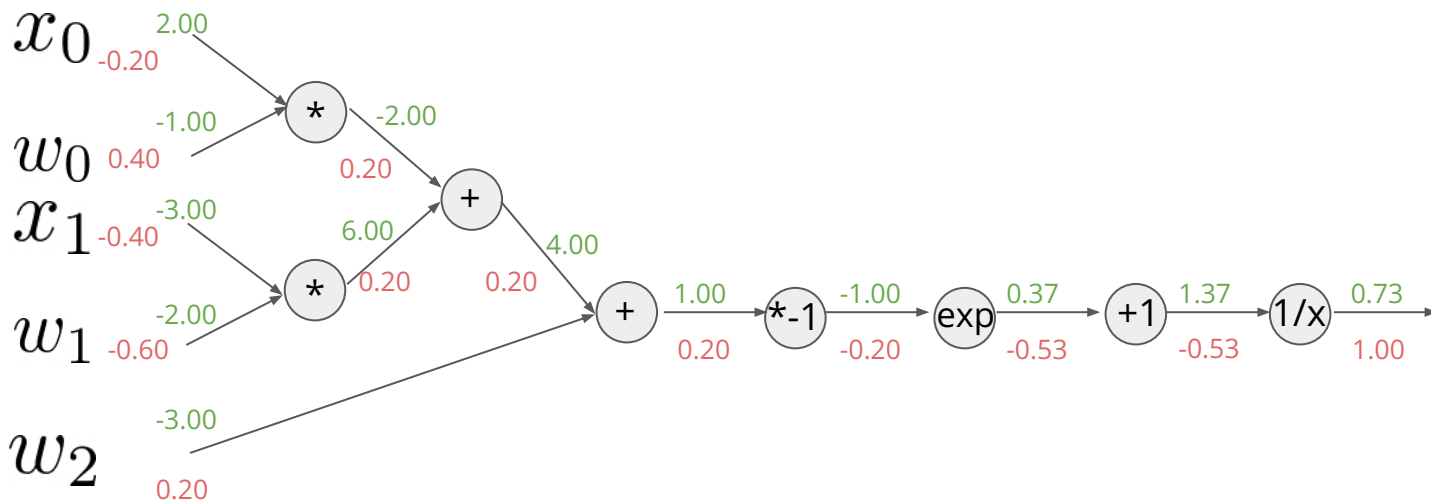
Backprop is just way to use it in NN training.





Backpropagation example

$$L(w, x) = \frac{1}{1 + \exp(-(x_0 w_0 + x_1 w_1 + w_2))}$$





Backpropagation: matrix form

$$\begin{aligned}y_1 &= f_1(\mathbf{x}) = x_1 \\y_2 &= f_2(\mathbf{x}) = x_2 \\&\vdots \\y_n &= f_n(\mathbf{x}) = x_n\end{aligned}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \nabla f_2(\mathbf{x}) \\ \dots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}} f_1(\mathbf{x}) \\ \frac{\partial}{\partial \mathbf{x}} f_2(\mathbf{x}) \\ \dots \\ \frac{\partial}{\partial \mathbf{x}} f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$



Backpropagation: matrix form

	vector	
scalar	x	$\mathbf{x}$
scalar	$\frac{\partial f}{\partial x}$	$\frac{\partial f}{\partial \mathbf{x}}$
vector	$\frac{\partial \mathbf{f}}{\partial x}$	$\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$



Backpropagation: matrix form

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}} f_1(\mathbf{x}) \\ \frac{\partial}{\partial \mathbf{x}} f_2(\mathbf{x}) \\ \dots \\ \frac{\partial}{\partial \mathbf{x}} f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

$$= \begin{bmatrix} \frac{\partial}{\partial x_1} x_1 & \frac{\partial}{\partial x_2} x_1 & \dots & \frac{\partial}{\partial x_n} x_1 \\ \frac{\partial}{\partial x_1} x_2 & \frac{\partial}{\partial x_2} x_2 & \dots & \frac{\partial}{\partial x_n} x_2 \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} x_n & \frac{\partial}{\partial x_2} x_n & \dots & \frac{\partial}{\partial x_n} x_n \end{bmatrix}$$

(and since $\frac{\partial}{\partial x_j} x_i = 0$ for $j \neq i$)

$$= \begin{bmatrix} \frac{\partial}{\partial x_1} x_1 & 0 & \dots & 0 \\ 0 & \frac{\partial}{\partial x_2} x_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & \frac{\partial}{\partial x_n} x_n \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

$= I$ (I is the identity matrix with ones down the diagonal)

Fancy neural networks

girafe
ai

06

RNN



Shakespeare

PANDARUS:

Alas, I think he shall be come approached and the day
When little grain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and
my fair nues begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

Algebraic Geometry (Latex)

Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. *This is an integer $\mathbb{Z}$ is injective.*

Proof. See Spaces, Lemma ?? □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset X$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b: X \rightarrow Y' \rightarrow Y \rightarrow Y' \times_X Y' \rightarrow X.$$

be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(\mathcal{U})$ which is locally of finite type. □

Linux kernel (source code)

```
/*  
 * If this error is set, we will need anything right after that BSD.  
 */  
  
static void action_new_function(struct s_stat_info *wb)  
{  
    unsigned long flags;  
    int lel_idx_bit = e->edd, *sys & -((unsigned long) *FIRST_COMPAT);  
    buf[0] = 0x7fffffff & (bit << 4);  
    min(inc, slist->bytes);  
    printk(KERN_WARNING "Memory allocated %02x/%02x, "  
           "original MLL instead\n"),  
           min(min(multi_run - s->len, max) * num_data_in),  
           frame_pos, sz + first_seg);  
    div_u64_w(val, inb_p);  
    spin_unlock(&disk->queue_lock);  
    mutex_unlock(&s->sock->mutex);  
    mutex_unlock(&func->mutex);  
    return disassemble(info->pending_bh);  
}
```



Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. *This is an integer Z is injective.*

Proof. See Spaces, Lemma ?? □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b: X \rightarrow Y' \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

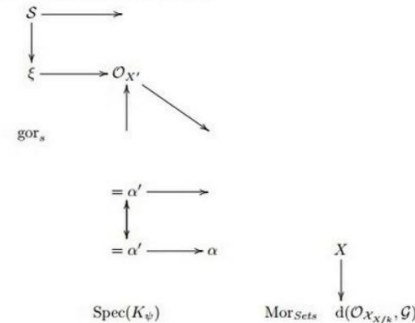
be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_* . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U . □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_x \longrightarrow \mathcal{O}_{X_t}^{-1} \mathcal{O}_{X_\lambda}(\mathcal{O}_{X_q}^v)$$

is an isomorphism of covering of $\mathcal{O}_{X_t}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S .

If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_\lambda}$ is a closed immersion, see Lemma ??.

This is a sequence of $\mathcal{F}$ is a similar morphism.



```
#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>

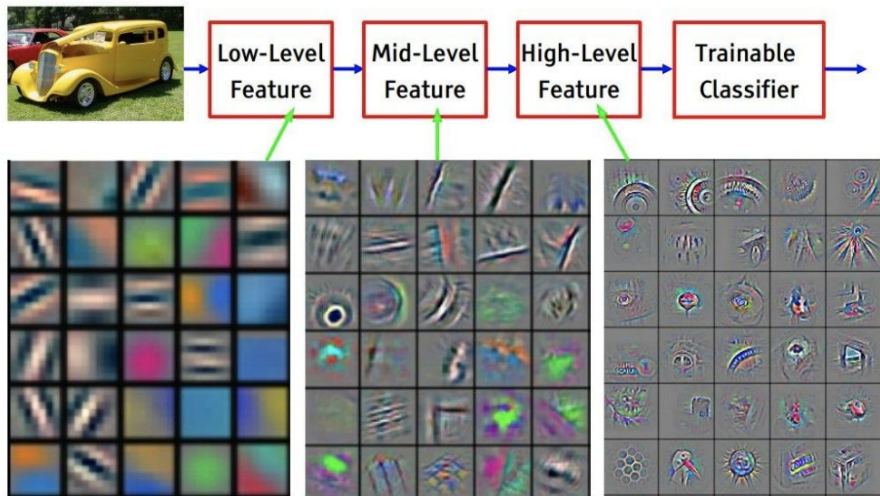
#define REG_PG      vesa_slot_addr_pack
#define PFM_NOCOMP  AFSR(0, load)
#define STACK_DDR(type)      (func)

#define SWAP_ALLOCATE(nr)      (e)
#define emulate_sigs()  arch_get_unaligned_child()
#define access_rw(TST)  asm volatile("movd %%esp, %0, %3" : : "r" (0)); \
    if (__type & DO_READ)

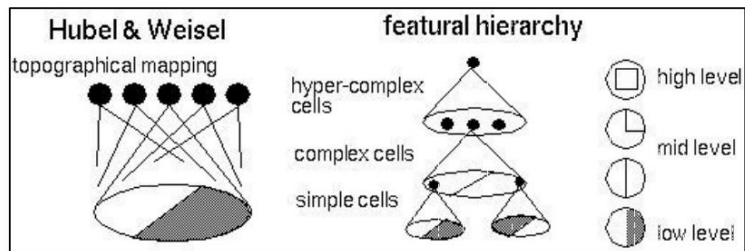
static void stat_PC_SEC __read_mostly offsetof(struct seq_argsqueue, \
    pc>[1]);

static void
os_prefix(unsigned long sys)
{
#ifdef CONFIG_PREEMPT
    PUT_PARAM_RAID(2, sel) = get_state_state();
    set_pid_sum((unsigned long)state, current_state_str(),
        (unsigned long)-1->lr_full; low;
}
}
```

CNN: Convolutional layer and visual cortex



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



[From Yann LeCun slides]

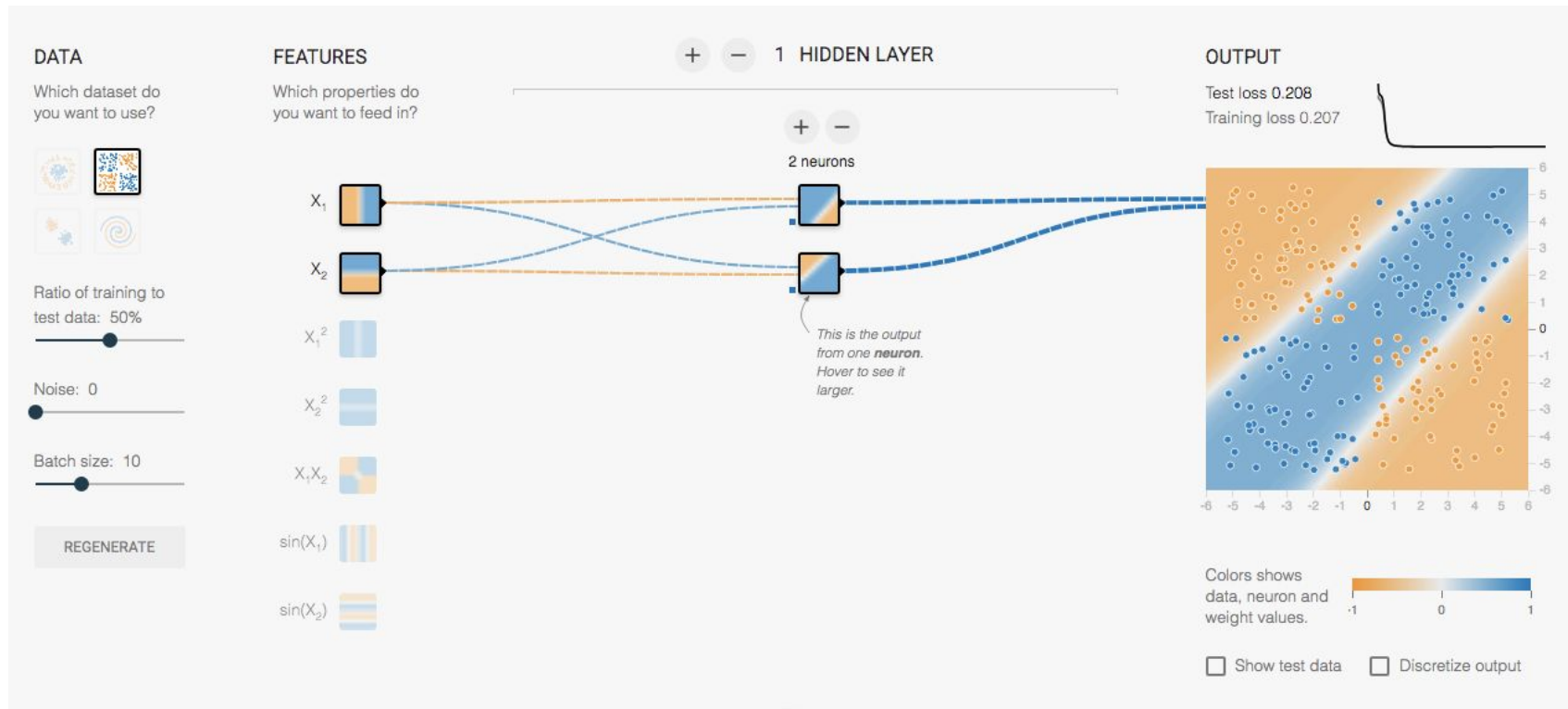
CNN:Convolutional layer and visual cortex



source



Don't miss the interactive playground



Neural network libraries overview

girafe
ai

07

Pytorch



King of the hill at the moment.

Originally developed by Facebook since 2016.

Written in C++, has 83k stars on Github (2024)

BSD-3 licensed



Tensorflow



Previous king of the hill.

Originally developed by Google since 2015.

Written in C++, has 186k stars on Github (2024)

Apache License 2.0



TensorFlow

Jax



<https://docs.jax.dev/en/latest/quickstart.html>

The next big thing.

Originally developed by Google since 2018.

Framework for neural networks training - [Flax](#)



Outro



- Neural Networks are great
 - Especially for data with specific structure
- All operations should be differentiable to use backpropagation mechanics
 - And still it is just basic differentiation
- Many techniques in Deep Learning are inspired by nature
 - Or general sense
- Do not hesitate to ask questions (and answer them as well)

More materials for self-study: [link](#)

Revise



1. Neural Networks in different areas.
Historical overview.
2. Backpropagation.
3. More on backpropagation.
4. Activation functions.
5. Playground.

Thanks for attention!

Questions?

